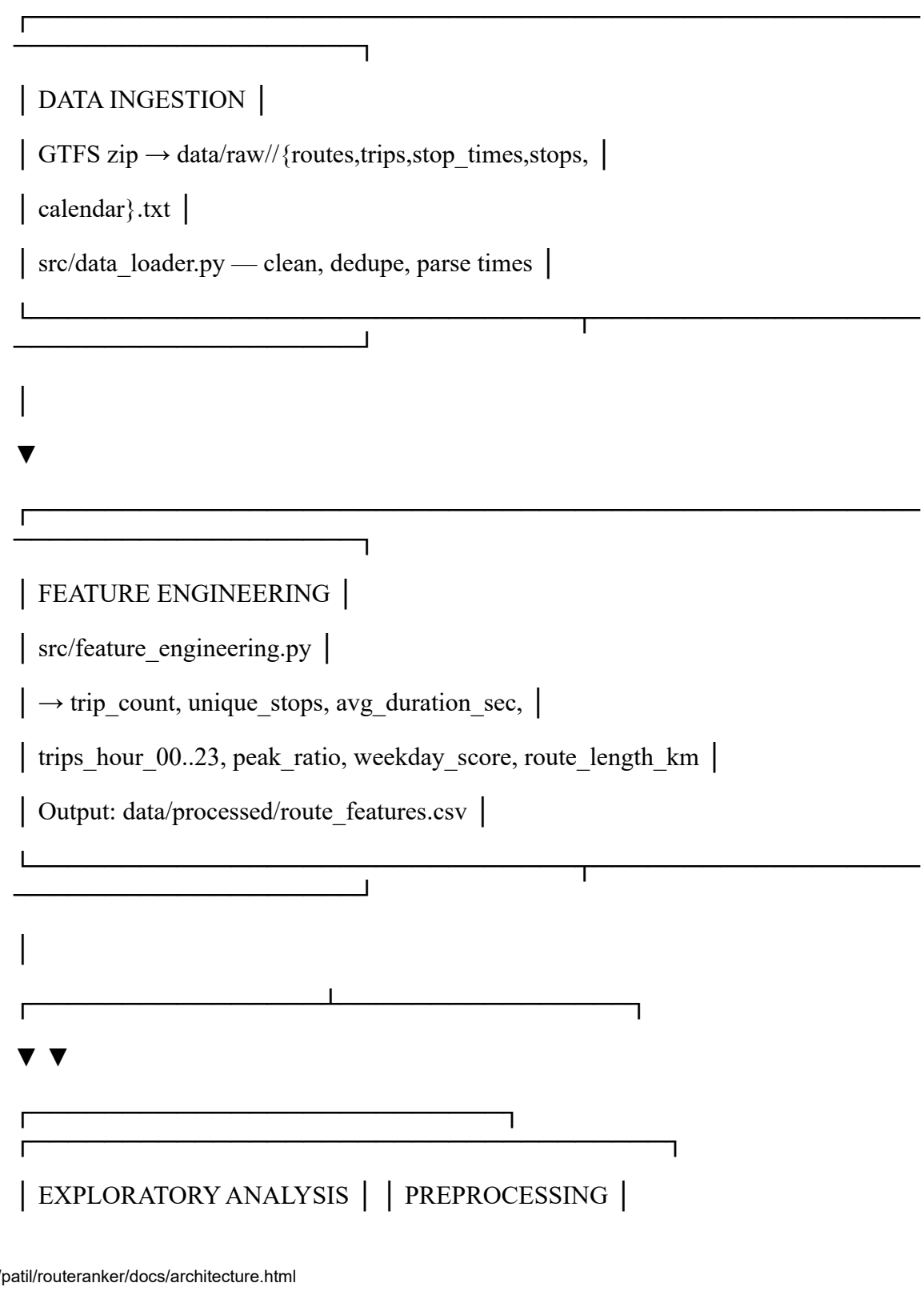
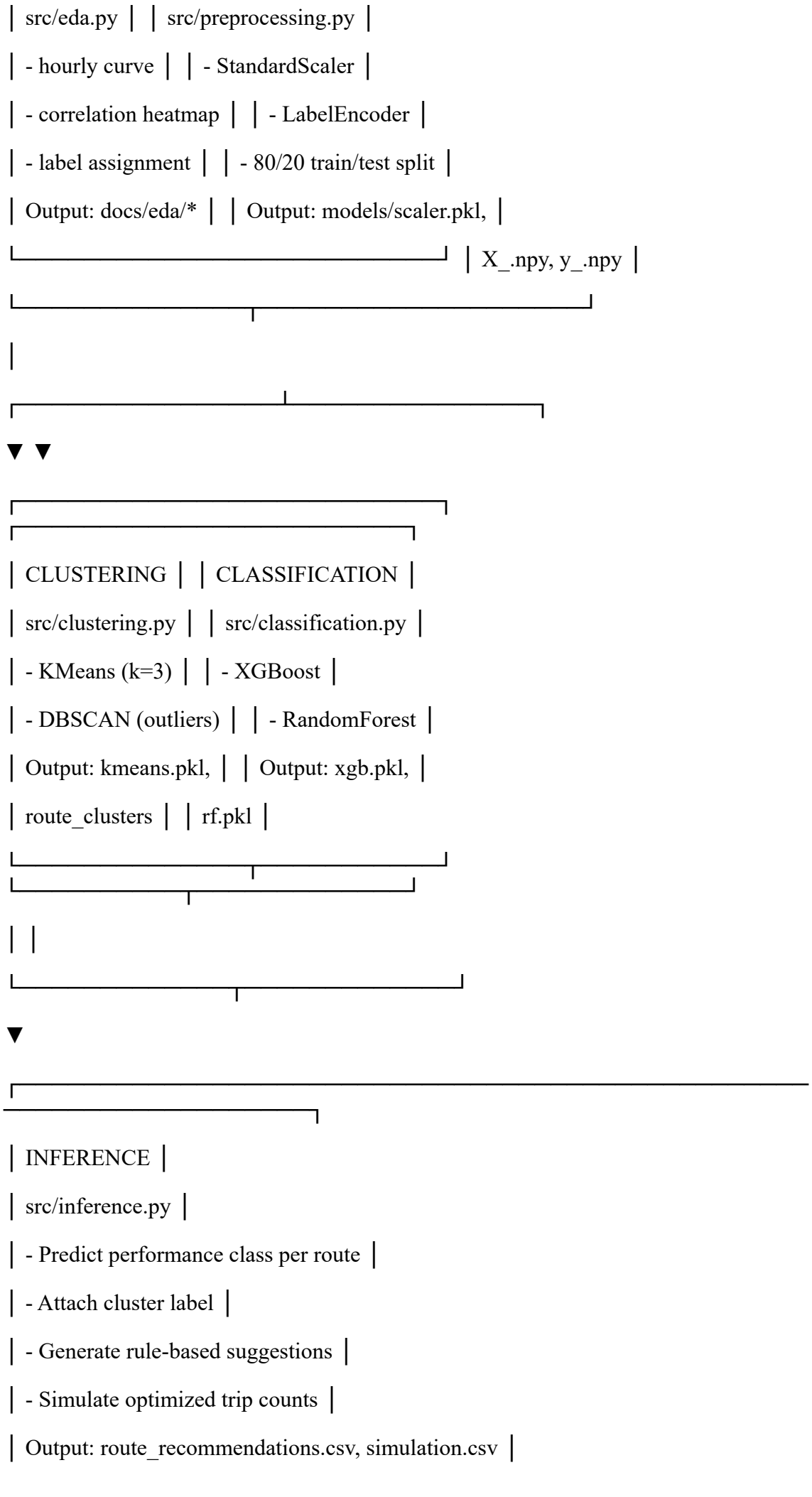
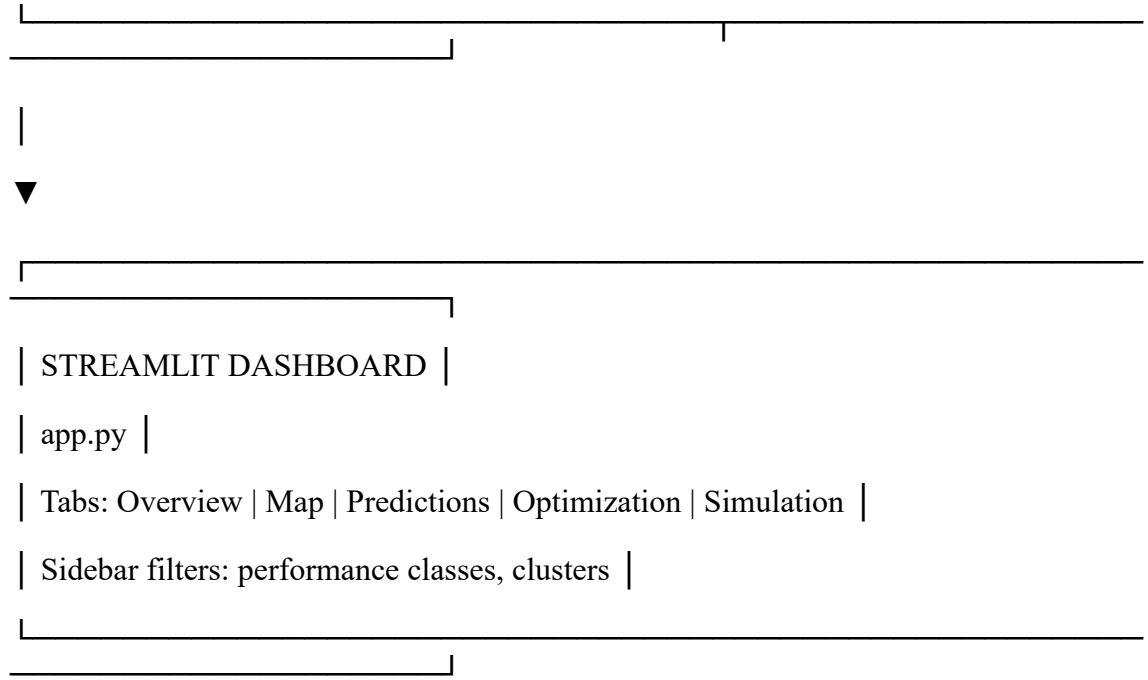
RouteRanker Architecture

RouteRanker — System Architecture

Component Diagram




src/eda.py
- hourly curve
- correlation heatmap
- label assignment
Output: docs/eda/*
src/preprocessing.py
- StandardScaler
- LabelEncoder
- 80/20 train/test split
Output: models/scaler.pkl,
X_.npy, y_.npy
CLUSTERING
src/clustering.py
- KMeans (k=3)
- DBSCAN (outliers)
Output: kmeans.pkl,
route_clusters
CLASSIFICATION
src/classification.py
- XGBoost
- RandomForest
Output: xgb.pkl,
rf.pkl
INFERENCE
src/inference.py
- Predict performance class per route
- Attach cluster label
- Generate rule-based suggestions
- Simulate optimized trip counts
Output: route_recommendations.csv, simulation.csv



Data Contracts

Stage	Input	Output
data_loader	raw GTFS .txt	cleaned DataFrames with `arrival_sec`
feature_engineering	cleaned DataFrames	1 row per route × ~30 columns
eda	route_features.csv	charts + rough_label column
preprocessing	route_features_labeled.csv	X_*.npy, y_*.npy, scaler.pkl, encoder.pkl
clustering	X_scaled.npy	kmeans.pkl, route_clusters.csv
classification	X_train/test.npy	xgb.pkl, rf.pkl
inference	all models + features	recommendations + simulation csv
app	all csv/pkl	Streamlit UI

Runtime

Stage	BMTC (4,416 routes)
data_loader	~90 seconds
feature_engineering	~90 seconds
eda	~5 seconds
preprocessing	~3 seconds
clustering	~15 seconds
classification	~30 seconds
inference	~5 seconds
Total	**~4 minutes**

Extension Points

- **New city** → drop GTFS files in `data/raw/<city>/`, change one `CITY` constant in `data_loader.py` and `feature_engineering.py`.
- **New feature** → add a column in `build_route_features()`; downstream scripts pick it up automatically.

- **New model** → implement in a new `src/<model>.py` that reads `X\_\*.npy`, writes a `.pkl`. Register in `inference.py`.

- **New dashboard tab** → add a `with tabN:` block in `app.py`.